

读懂 Scheduler 的五个初始化

从算法、状态与对象关系进入代码，而不是逐行遍历三千行。

阅读范围与证据

以当前工作区 `third_party/sglang` 的源码快照为准。仓库 HEAD 为 **5d6bef9f61637aaeaf047bf8209def2af3eaa83f**，`scheduler.py` 存在本地修改；本文依据实际文件而非仅依据提交版本。源码快照与哈希保存在生成目录的 `source_snapshot` 中。

主线假设：单卡 CUDA、普通 Qwen3 文本自回归生成、无推测解码、无 PD 分离、无 LoRA 和多模态请求。是否启用 `radix`、`chunked prefill`、`overlap` 仍由实际参数决定，本文给出判断入口。没有运行模型或读取你的启动参数，因此不把默认值当作已验证的运行值。

初始化方法	建立的抽象	正文页
<code>init_running_status</code>	请求集合与调度循环状态	2-3
<code>init_model_worker</code>	执行器引用、资源容量和通信上下文	4-5
<code>init_cache_with_memory_pool</code>	请求到 KV 的映射、分配器、前缀索引	6-8
<code>init_schedule_policy</code>	排序策略和未来 token 容量估计	9-10
<code>init_overlap</code>	stream 上下文、未来 token 映射、引用保活	11-12

初始化与算法的边界

这五个方法均没有显式返回值，正常返回 `None`。它们通过修改 `self`、共享对象或少量全局状态完成初始化。算法通常在其建立的对象上、收到请求之后才执行；但 `init_model_worker` 会进入模型加载和执行环境初始化，`init_overlap` 也可能分配设备 `buffer`。

实际调用顺序是 `model_worker` → `cache_with_memory_pool` → `running_status` → `schedule_policy` → `overlap`，中间穿插 `chunked prefill` 等其他初始化。先有 `worker` 提供内存池，再由 `Scheduler` 连接缓存与策略。

如何使用本文：先读每节的成员表，再看一个数值或状态例子，最后只在列出的消费者方法上打断点。第 13 页提供单请求检查表。

源码定位：[S1] `init_model_worker()`，L567；[S1] `init_cache_with_memory_pool()`，L640；[S1] `init_running_status()`，L753；[S1] `init_schedule_policy()`，L808；[S1] `init_overlap()`，L1011。文件路径见第 14 页。

02 / init_running_status

01 请求集合与运行状态

这个方法创建一个尚未接收请求的状态机。它没有排序、模型计算或 KV 分配算法；它为连续批处理和控制请求提供后续可更新的字段。

成员 / 初始值	准确含义与运行时用途
waiting_queue = []	List[Req]。普通新请求先入队；未获本轮准入的请求留下。被撤回的请求也可重新进入。不是只有过载才使用。
running_batch = ScheduleBatch(reqs=[], batch_is_full=False)	持续维护的 decode batch。内部 reqs 是请求列表，同时持有序列长度、地址映射等 batch 状态。batch_is_full 是准入提示，不等同于 GPU 利用率已满。
cur_batch = None	本轮实际选中的 forward batch 引用。可能是 prefill batch、running_batch，也可能为 None。
last_batch = None	上一轮选中的 batch 引用。用于跨轮合并 prefill 请求、协调 overlap 结果处理及判断相邻 batch 类型；不是公平性历史表。
forward_ct = 0	run_batch 调用时递增，用于执行计数、profiling 等。不是 GPU kernel 数，也不是输出 token 数。
return_health_check_ct = 0	忙碌时暂缓响应的 health-check generation 请求计数；后续发送健康响应时扣减。
num_retracted_reqs = 0	记录撤回相关数量。普通内存不足路径写入本次撤回列表长度；不能直接当作终身累计量。
num_paused_reqs = 0	暂停请求计数字段。当前 managers 路径检索只发现初始化，学习普通推理时无需赋予额外调度语义。
sessions = {}	session ID 到 Session 的映射。普通独立请求可先忽略；不是 KV 前缀缓存本身。
forward_sleep_time = None	人为延迟 forward 的调试控制项，run_batch 读取。不是空闲等待策略。
_engine_paused = False	暂停开关。事件循环处理输入后检查它；暂停时跳过后续正常调度。

不在这个方法初始化：chunked_req 在 init_chunked_prefill 中；result_queue 在进入 event_loop_overlap 时创建。不要把所有队列都归到 init_running_status。

源码定位：[S1] init_running_status(), L753；[S1] run_batch(), L2368；[S1] maybe_send_health_check_signal(), L2556。文件路径见第 14 页。

为什么既有 running，又有 cur / last

集合维度：running_batch 主要维护 decode 请求。**轮次维度：**cur_batch、last_batch 记住当前与上一轮选中的工作。这两个维度不能互相替代。

场景	running_batch	cur_batch
A 已在 decode，B 新到且本轮选 B prefill	包含 A	包含 B 的新 prefill batch
后续 B 被合入，选 A/B 一起 decode	包含 A、B	可以直接引用 running_batch
没有可运行工作	通常为空 batch	None

你的“两条队列”设计依然成立

如果每轮执行完就消费本轮结果、合并完成 prefill 的请求、移除完成请求，下一轮可以仅依据更新后的 waiting/running 集合调度，不必有独立 last_batch。SGLang 把 prefill 合并放在下一轮 get_next_batch_to_run；overlap 又延后结果处理，所以保留跨轮引用。

```
# 同步式教学设计：收尾在本轮完成
batch = choose(waiting, running)
result = run_and_wait(batch)
update_requests(batch, result)
merge_prefill_and_remove_finished(running, batch)

# SGLang 的组织方式（省略细节）
batch = get_next_batch_to_run() # 内部使用 last_batch 合并
self.cur_batch = batch
# 提交当前批、消费先前结果.....
self.last_batch = batch
```

对象引用不是请求副本

cur_batch = running_batch、last_batch = batch 都只是 Python 引用赋值。同一个 Req 可以被多个 batch 视图引用。last_batch 不是不可变的历史快照；result_queue 使用的 batch.copy() 也只保留后处理需要的字段，并共享 Req 引用，不能理解为完整深拷贝。

算法视角

连续批处理的状态变化为：等待 → 准入并 prefill → 持续 decode → 完成移除；KV 不足还可撤回到等待。一次 forward batch 通常只推进一个阶段或一步，而不是把整批请求的回答全部生成完。

源码定位：[S1] get_next_batch_to_run(), L1939; [S1] event_loop_overlap(), L1156; [S11] copy(), L2281; [S11] filter_batch(), L2077。文件路径见第 14 页。

04 / init_model_worker

02 把模型执行器接到调度器

这个方法建立“谁来执行”的连接，再把执行器测得或配置的容量上限同步到 Scheduler。它本身不进行请求排序，也不返回某个请求的 logits。



无推测解码：model_worker 与 tp_worker 指向同一个对象。
对象调用关系；不表示三个进程，也不表示三张 GPU。

成员 / 设置	来源与意义
tp_worker	init_tp_model_worker 创建 TpModelWorker。名字带 TP，但 tp_size=1 时仍使用这个类。不能据此认定启用了多卡计算。
draft_worker	maybe_init_draft_worker 中，无推测解码时设为 None；启用时才创建相应 draft worker。
model_worker	无推测解码时直接等于 tp_worker；否则指向 draft_worker。后续 run_batch 通过它的 forward_batch_generation 发起执行。
device	从 worker 返回的设备标识；CUDA 路径后续用它选择 torch 设备模块。
forward_stream	从 worker.model_runner 获取的 stream 引用，并非由 init_overlap 新建。
random_seed	worker 返回的随机种子；本方法调用 set_random_seed 更新相关 RNG 状态。
pad_input_ids_func	worker 提供的多模态 padding 回调，模型未提供时可为 None。普通文本路径不依赖它。

真正较重的初始化发生在哪里？

构造 TpModelWorker 会进入 ModelRunner：模型配置、权重加载、设备运行环境、内存池及相关执行准备都在这个层次展开。理解 scheduler 时，只需知道 worker 返回了可用执行入口和哪些资源上限；量化或具体 attention kernel 的实现可留在 ModelRunner 内部。

若实际模型是 Qwen3-30B-A3B 等 MoE 变体，单卡不意味着没有专家计算；EP 的多卡分布逻辑可跳过，MoE 层内部计算仍可能存在。这不会改变这里的请求调度接口。

源码定位：[S1] init_model_worker(), L567；[S1] init_tp_model_worker(), L523；[S1] maybe_init_draft_worker(), L538；[S2] _init_model_runner(), L327；[S2] get_worker_info(), L394。文件路径见第 14 页。

05 / init_model_worker / 完整成员补充

执行容量与辅助成员

容量成员	单位 / 来源 / 用法
max_total_num_tokens	KV token 槽位容量；来自 ModelRunner。不是模型一次输出 token 的上限，也不是可空闲分配数量。
max_prefill_tokens	本批 prefill 输入 token 预算参数；来自 server_args，后续由 PrefillAdder 使用。不要把所有分支简化为绝对的硬截断。
max_running_requests	同时接纳请求的数量上限；来自 ModelRunner。独立于 token 容量，两个约束都需满足。
max_queued_requests	等待队列容量限制，来自 server_args；None 表示该配置项未限制。
max_req_len	当前 worker 中为 min(context_len - 1, max_token_pool_size - 1)。限制单请求总长度。
max_req_input_len	当前 worker 中为 max_req_len - 5，保留实现需要的余量；输入校验使用。

通信上下文也会初始化，但单卡不必展开 collective

成员	作用
tp_group / tp_cpu_group	基础 TP group 及其 CPU 通信组引用。
attn_tp_group / attn_tp_cpu_group	attention TP group 及 CPU 组。
attn_cp_group / attn_cp_cpu_group	attention CP group 及 CPU 组。
pp_group / world_group	PP 和全局 group 引用。单卡仍可存在 world_size=1 的统一接口。
dp_tp_group / dp_tp_cpu_group	请求协调所用 group；enable_dp_attention 时选 attn_tp_group，否则选 tp_group。不是新增一个模型副本。

其他副作用：全局 server_args.pp_max_micro_batch_size 若为 None，会被设置为 max(max_running_requests // pp_size, 1)；还可能输出日志和 cache 配置指标。get_worker_info 返回的最后三个 pool 尺寸在这里用 _ 丢弃，没有成为新的 Scheduler 成员。

容量数值应从运行时读

显卡型号和“30B”不足以推出这些值：权重量化、KV dtype、上下文长度、attention 结构和静态显存比例都会影响它们。调试时直接记录这里返回的容量；KV 容量的计算关系见第 8 页。

源码定位：[S2] get_worker_info(), L394；[S1] init_model_worker(), L567；[S4] init_memory_pool(), L468。文件路径见第 14 页。

06 / init_cache_with_memory_pool

03 KV 缓存：三个对象，三种职责

这个方法首先从 worker 获取已有内存池引用，再创建适配这些池的 cache 管理器。方法名中的 init 不意味着在这里重新分配一套模型 K/V 张量。



前缀缓存选择与其他初始化成员

成员	初始化与适用性
is_hybrid_swa	从 tp_worker 获取；用于区分滑动窗口与 full attention 混合池。
is_hybrid_ssm	根据 ModelRunner.hybrid_gdn_config 或 mamba2_config 是否存在设置。
sliding_window_size	先为 None；hybrid SWA 时取 worker 的窗口大小。
full_tokens_per_layer / swa_tokens_per_layer	仅 hybrid SWA 分支赋值，来自 get_tokens_per_layer_info；当前实现返回 full/SWA 最大 token 容量，不应按名字误读成层数。
hisparse_coordinator	仅 enable_hisparse 时引用 ModelRunner 中的协调器。
decode_offload_manager	只有 PD decode 且启用 KV offload 才创建；否则 None。

局部 params 是 CachelnitParams，不是 self.params。它打包池引用、page_size、禁用标志、驱逐策略、通信组等。方法末尾 init_mm_embedding_cache 会初始化全局多模态 embedding cache 对象；不等于普通文本模型在此计算 embedding。

tree_cache 的分支优先级

首先：chunked_prefill_size 配置非 None 且 disable_radix_cache 时，选 ChunkCache（或 SWAChunkCache）。否则依次检查实验 C++ radix → hierarchical cache → hybrid SWA → hybrid SSM → LMCache → 普通 RadixCache。即使最后类型是 RadixCache，其 disable 参数仍可能为 True。

Radix 算法在这里对应哪些操作？

查找与插入：以 token 前缀为 key，压缩树节点保存对应的 KV 索引；最长公共前缀匹配提供可复用位置，插入时按公共前缀拆分节点。命中只减少需要重新计算的后缀，实际截断还受 page 对齐和输出需求影响。

保护与驱逐：活跃请求对节点增加 lock_ref；释放使用权后节点可变为可驱逐。空间紧张时从可驱逐叶节点按策略回收，默认配置是 LRU。因此“请求结束”不等于“它的 KV 全部立即清空”。

ChunkCache：用于在关闭跨请求 radix 复用时保存请求自身的分块进度，不应把它理解为仍然完整提供跨请求最长前缀共享。

源码定位：[S1] init_cache_with_memory_pool(), L640；[S7] match_prefix(), L352；[S7] insert(), L424；[S7] evict(), L565；[S17] init_mm_embedding_cache(), L378。文件路径见第 14 页。

08 / 把 memory pool 和准入约束连接起来

KV 容量：从显存字节到调度预算

普通 full-attention、无特殊压缩 KV 的单卡模型，每个 token 跨所有 attention 层需要的 KV 字节数可写为：

```
bytes_per_token = L * H_kv * (D_k + D_v) * bytes_per_element
# 若 D_k = D_v = D:
bytes_per_token = 2 * L * H_kv * D * bytes_per_element
```

L 是实际计算 KV 的层数，H_kv 是本设备 KV head 数，D_k/D_v 是 K/V head dimension。GQA 应使用 KV head 数，而不是 query head 数。量化 KV 的 scale buffer、混合窗口、MLA 等会改变公式；权重量化位宽也不自动等于 KV 位宽。

当前代码的容量估计

```
# 单位: available / total_memory 在源码中按 GiB 表示
rest_memory = available_gpu_memory - total_gpu_memory * (1 - f)
capacity = int(rest_memory * 2**30) // bytes_per_token
# f = mem_fraction_static
# 后续还可能受 max_total_tokens、page 对齐和其他配置调整
```

这里的 available_gpu_memory 是 profiling 时刻的可用显存，已受模型加载等占用影响。因此不能直接用 “显卡总显存 × f” 作为 KV 预算。池通常按容量预分配，allocator 再在池内分配位置。

示例：仅演示量纲，不是你的 Qwen3 配置

假设 L=32、H_kv=8、D=128、KV 为 2 字节元素，则每 token 的 KV 占用为 131,072 字节，即 128 KiB。若为 KV 留出 4 GiB，未计额外开销和调整时，理论容量为 32,768 个 token 槽位。

约束	回答的问题
max_total_num_tokens	整个 KV 池最多容纳多少 token 位置？
allocator.available_size()	此刻池里有多少空闲位置？
tree_cache.evictable_size()	缓存中有多少未锁定位置可以通过驱逐回收？
req_to_token_pool.available_size()	还能多少个新请求分配映射表行？
max_prefill_tokens / chunk budget	本轮愿意执行多少 prefill 工作？

即使还有空闲请求槽位，也可能没有足够 KV；即使有 KV，也可能因本轮 prefill 预算不足而等待。这就是 “两条队列” 之外必须存在资源元数据的原因。

源码定位：[S4] get_cell_size_per_token(), L57; [S4] profile_max_num_token(), L164; [S4] init_memory_pool(), L468; [S9] rem_total_tokens(), L451。文件路径见第 14 页。

04 排序策略与未来 token 估计

这个初始化同时设置两类东西：等待队列的候选顺序，以及为已有 decode 请求预留多少未来 KV 容量。两者分别对应排序和准入估计，不能混为公平性权重。

成员	初始化方式与消费者
policy	创建 SchedulePolicy(schedule_policy, tree_cache, ...)。后续 calc_priority 原地调整 waiting_queue。schedule_policy 字符串本身在更早配置阶段已有。
prefill_delayer	默认 None；enable_prefill_delayer 时建立延后 prefill 的协调对象。本单卡基础路径先忽略其内部算法。
try_preemption	设为 enable_priority_scheduling。它表示是否尝试基于优先级抢占；不是所有内存不足撤回行为的总开关。
init_new_token_ratio	$r0 = \min(\text{INIT_RATIO} \times \text{schedule_conservativeness}, 1)$ 。
min_new_token_ratio	$rmin = \min(r0 \times \text{MIN_RATIO_FACTOR}, 1)$ 。
new_token_ratio_decay	$\text{delta} = (r0 - rmin) / \text{DECAY_STEPS}$ 。
new_token_ratio	初始化为 r0；每次构造 PrefillAdder 时传入，参与已有请求未来 token 的容量预估。

默认数值与变化方式

当前默认环境值为 INIT_RATIO=0.7、MIN_RATIO_FACTOR=0.14、DECAY_STEPS=600；schedule_conservativeness 默认 1.0。因此 r0=0.7、rmin=0.098、delta≈0.00100333。这些是源码默认值，环境变量与启动配置可以覆盖。

```
# 普通 update_running_batch 的未撤回分支
new_token_ratio = max(new_token_ratio - delta, rmin)

# 空闲自检满足条件时
new_token_ratio = r0
```

发生 decode 撤回时，retract_decode 根据剩余请求已生成 token 和预留步数重新估计 ratio，Scheduler 使用返回值。不要把它写成“每轮固定衰减”或“遇到 OOM 固定加某个数”。

算法解释

较小 ratio 为已有请求估计更少的未来增长，通常允许更多新请求进入；较大 ratio 更保守。这不是精确输出长度预测，也不改变请求的 max_new_tokens 结束条件。实际准入还要同时检查待加入请求自己的需求。

源码定位：[S1] init_schedule_policy(), L808；[S1] update_running_batch(), L2293；[S11] retract_decode(), L1876；[S14] self_check_during_idle(), L411。文件路径见第 14 页。

10 / SchedulePolicy 与 PrefillAdder 的分工

排序不等于准入：一个预算例子

策略	本地实现的主要行为
FCFS（默认）	无显式优先级时保留等待列表顺序；开启优先级则先按优先级、再按入队时间排序。
LPM	优先已有缓存匹配前缀更长的请求；包含等待队列内部共享前缀的暂缓逻辑。队列大于 128 时退回 FCFS。
DFS_WEIGHT	在缓存树上统计请求子树权重，按权重组织深度优先遍历，使共享前缀请求在顺序上靠近。
LOF / RANDOM / ROUTING_KEY	分别按请求的 max_new_tokens 降序、随机排列、或运行 batch 中 routing key 频率等规则排序。LOF 不使用真实未来输出长度。

缓存感知策略在 tree_cache 被禁用时会调整为 FCFS。即使普通 FCFS 不主动按命中率排序，后面的请求准备仍可使用 radix 前缀复用；“FCFS”不等于“关闭 prefix cache”。

已有 decode 请求先占用未来预算

```
reserve_running = sum(
    min(req.max_new_tokens - len(req.output_ids), clip) * ratio
    for req in running_batch.reqs
)
remaining = free_kv + evictable_kv - reserve_running
# 教学写法: req.max_new_tokens 实际位于 req.sampling_params
# clip 的本地默认值为 4096，只裁剪估计，不裁剪输出上限
```

例：可用可驱逐空间为 10,000；A、B 剩余最大输出分别 1,000、2,000；ratio=0.7。已有请求的未来预算为 2,100，剩余预算为 7,900。这里假设普通缓存、没有额外混合 batch offset。

候选请求不是统一乘 ratio 后就能入场

新请求 C 若尚需 prefill 2,000 token，剩余最大输出 1,000，page_size=1，普通 add_one_req 的初步需求为 2,000+1,000+1=3,001，小于 7,900，才继续检查输入预算、前缀锁定后的空间等。完整 prefill 接纳后，普通预算更新再扣去输入和裁剪后的输出预算。

因此不能把整个算法简化成“所有请求的 max_new_tokens 都乘 ratio”。当前源码对已有 running 请求与待加入候选使用不同步骤；分块、ignore_eos、page 对齐还有专门分支。排序决定先尝试谁，PrefillAdder 决定本轮装得下谁。

源码定位：[S9] calc_priority(), L114；[S9] _determine_active_policy(), L158；[S9] _get_running_request_total_token_offset(), L441；[S9] add_one_req(), L744；[S9] _update_prefill_budget(), L526。文件路径见第 14 页。

11 / init_overlap

05 为 CPU / GPU 重叠准备状态

目标是让 CPU 准备下一批、处理上一批结果的时间，与 GPU 已提交的计算重叠。它不意味着同一请求的相邻 token 可以无视数据依赖，也不保证两个模型 forward 在 GPU 上同时运行。

成员	来源、初值与作用
device_module	torch.get_device_module(device)。CUDA 时提供 stream、Event 等接口。
default_stream	记录初始化当时 current_stream(); 名字不保证它必然是 CUDA legacy default stream。CPU 路径将其 synchronize 替换为空操作。
forward_stream_ctx	由已有 forward_stream 构建的上下文管理器，进入后把相关操作提交到该 stream。forward_stream 来自 init_model_worker。
copy_stream / copy_stream_ctx	这里创建独立 stream 及上下文。当前普通 run_batch 的结果拷贝在 forward_stream_ctx 内提交；不能仅凭字段名认定所有 D2H 都走 copy_stream。PP 路径可使用它。
future_map	关闭 overlap 时设为 None 并返回；开启时创建 FutureMap，普通非推测解码用设备 int64 buffer 存未来 token ID。
batch_record_buf = [None, None]	仅 overlap 分支创建。轮换保留 ModelWorkerBatch 引用，避免异步 GPU 使用的数据过早失去 Python 引用。
batch_record_ct = 0	仅 overlap 分支创建。每次 record_batch_in_overlap 按模 2 更新，选择引用保活槽位。

两个容易混淆的“buffer”

batch_record_buf 服务于对象/张量生命周期；result_queue 服务于结果后处理，两者不是同一个队列。result_queue 在 event_loop_overlap 开头才初始化。FutureMap 则服务于下一步输入的数据依赖，保存的是 token 或推测解码中间数据。

关闭 overlap 时，也会执行前半段初始化

device_module、当前 stream、forward 上下文和 copy stream 的设置发生在 enable_overlap 判断之前。因此“init_overlap 被调用”不能作为 overlap 已启用的证据；应检查 enable_overlap 和 future_map。

源码定位：[S1] init_overlap(), L1011；[S1] record_batch_in_overlap(), L2359；[S1] run_batch(), L2368；[S10] alloc_future_indices(), L111；[S13] copy_to_cpu(), L52。文件路径见第 14 页。

12 / 保持 GPU 依赖，同时延后 CPU 后处理

FutureMap 如何绕开 CPU token 回传依赖

假设 A 正在生成 token t。CPU 准备下一步时还没拿到 t 的值，但可以先保存 future 槽位 j 的引用。下一次 GPU forward 前，再在设备上把占位符解析成真实 token ID。

```
# 非推测解码路径的教学表达
j = future_map.alloc_future_indices(batch_size).indices
# 本轮 GPU 采样得到 token_ids 后:
future_map.token_ids_buf[j] = token_ids
# 下一轮的输入先用负数编码 future 槽位:
next_input_ids = -j
# forward stream 上解析:
input_ids = where(input_ids < 0,
                  token_ids_buf[clamp(-input_ids, min=0)], input_ids)
```

负数在这里不是有效词表 token ID；它是 future 索引编码。解析发生在 forward stream 上，遵循先前 store 的顺序。forward_stream.wait_stream(default_stream) 又确保读取 batch 前，其依赖的准备操作已完成；这通常是设备侧排序，不是让 CPU 等待整张卡。

调度轮次	CPU 提交 / 状态准备	CPU 结果处理
第 1 轮	提交 A 的 prefill；保存 future 引用	没有上一批
第 2 轮	利用上轮 batch 合入 A，准备并提交 decode	消费 prefill 结果，将首 token 写入 Req.output_ids
第 3 轮	准备并提交下一批 decode	消费上一批 decode 结果，更新结束状态

这是无特殊禁用分支的顺序示意。process_batch_result 在读取 CPU 结果前会等待 copy_done；因此仍可能同步等待。连续 prefill 等情况也可提前处理上一批结果，实际以 is_disable_overlap_for_batch 为准。

环形空间与引用保活

设 $R = \max_running_requests$ ，分块数 $N = \lceil context_len / chunked_prefill_size \rceil$ ，未启用分块时 $N=0$ 。FutureMap 设 $future_limit = R \times (3 + N)$ ，buffer 长度再加 $2R$ ；索引按 batch 大小推进并环绕。这是当前实现的保守容量安排，不是通用公式。

batch_record_buf 的两个引用槽防止异步消费中的 batch 数据过早释放；result_queue 中的 batch.copy() 保存后处理需要的字段。GPU 可以已提交下一步，而 CPU 才发现上一结果结束请求，后处理会检查 finished/retracted，避免把多余结果继续追加。

源码定位：[S10] FutureMap._init_(), L36；[S10] resolve_future(), L120；[S10] store_to_map(), L151；[S1] event_loop_overlap(), L1156；[S12] process_batch_result_decode(), L417。文件路径见第 14 页。

13 / 调试检查表与阅读边界

用一个请求验证这五个抽象

准备一个短 prompt, max_new_tokens=4, 无 grammar/多模态/session。短输入让你先避开多轮 chunked prefill; EOS 可能使请求提前结束, 这是正常行为。断点放在赋值后的下一行, 或 Step Over 后再观察。

位置	应检查的对象与不变量
init_model_worker 末尾	无推测解码时: model_worker is tp_worker; draft_worker is None。 记录 max_total_num_tokens、max_running_requests、max_req_len。
init_cache_with_memory_pool 末尾	记录 type(tree_cache).__name__、tree_cache.disable、page_size; 确认 req_to_token_pool 与 worker 返回对象是同一个引用。
init_running_status 末尾	waiting_queue=[]; running_batch.reqs=[]; cur_batch/last_batch=None。尚未生成任何 token。
init_schedule_policy 末尾	看 policy.policy 的实际枚举, 以及 r0、rmin、decay。不要只看 server_args 的默认字符串。
init_overlap 末尾	看 enable_overlap、future_map 是否为 None; result_queue 此时尚不一定存在。
_add_request_to_queue 入队之后	确认目标 rid 已在 waiting_queue, Req.output_ids 仍为空。
_get_new_batch_prefill_raw	看 adder.can_run_list、移出等待队列后的内容、prepare_for_extend 后的 forward_mode 和地址字段。
run_batch / process_batch_result	按 rid 跟踪本轮 batch、future 输入、CPU output_ids 的增长。别将 forward 提交完成等同于 GPU 计算完成。
update_running_batch / filter_batch	看请求从 prefill 合入持续 decode, 结束后被移出; 缓存内容可能仍保留供复用。

在 Scheduler 栈帧中使用的 Watch

```
[r.rid for r in self.waiting_queue]
[(r.rid, len(r.output_ids)) for r in self.running_batch.reqs]
type(self.tree_cache).__name__
self.token_to_kv_pool_allocator.available_size()
self.req_to_token_pool.available_size()
self.policy.policy
self.new_token_ratio
```

记录请求状态、物理容量和 batch 模式通常已足够。初始化中不影响这些观察的可选功能可以 Step Over。不要在 GPU 异步执行时随意对大量 tensor 调用 .cpu() / .tolist(), 这些调试求值可能引入同步, 改变你观察到的重叠时序。

源码索引与可复核边界

所有相对路径均以 `third_party/sglang/python/sglang/srt/` 为根。正文中的 [S#] 与函数名是主要定位键；L 行号来自生成时快照，继续编辑后请优先按函数名搜索。

编号	文件路径
S1	managers/scheduler.py
S2	managers/tp_worker.py
S3	model_executor/model_runner.py
S4	model_executor/model_runner_kv_cache_mixin.py
S5	mem_cache/memory_pool.py
S6	mem_cache/allocator.py
S7	mem_cache/radix_cache.py
S8	mem_cache/chunk_cache.py
S9	managers/schedule_policy.py
S10	managers/overlap_utils.py
S11	managers/schedule_batch.py
S12	managers/scheduler_output_processor_mixin.py
S13	managers/utils.py
S14	managers/scheduler_runtime_checker_mixin.py
S15	server_args.py
S16	environ.py
S17	managers/mm_utils.py
S18	managers/scheduler_pp_mixin.py

五个入口的成员清单如何使用

第 2 页列出 `init_running_status` 的全部直接赋值；第 4-5 页覆盖 `init_model_worker` 及其创建 `worker` 的直接子调用；第 6-7 页覆盖 `cache` 初始化的普通与条件成员；第 9 页覆盖 `policy` 的全部直接成员；第 11 页覆盖 `overlap` 的全部直接成员。未把 `ModelRunner` 内所有嵌套初始化成员展开成第二份代码目录。

本文没有执行 GPU 推理或测量吞吐/延迟。容量算例为说明量纲，预算算例为说明代码决策；均不代表你的 4090 配置实测值。源码默认 FCFS/LRU/ratio 等可能被启动参数覆盖。PDF 已进行文本与页面渲染检查。

最后的复习问题：请求存在哪里？执行器是谁？KV 地址由谁分配和共享？先尝试谁、资源够不够分别由谁决定？CPU/GPU 的相邻两轮如何传递 token 和结果？能回答这五个问题，就可以沿着实际请求继续读源码。